

1. Single-stage vs. multi-stage Docker

The naive Dockerfile is one `python:3.11` stage. It copies everything in, installs the runtime and training requirements together, and trains the model during the build. The multi-stage version uses three stages. `deps` builds a virtualenv with only the runtime requirements, `trainer` adds pandas and makes `model.joblib`, and `runtime` starts from `python:3.11-slim` with just the venv, the model and `main.py`.

<code>spam-api:naive</code>	2.34 GB
<code>spam-api:multi</code>	645 MB
Reduction	72.4%

Four things stay behind in the builder stages: the base image, pandas, the dataset and training script, and pip's wheel cache.

What makes it work is that the runtime stage copies `/opt/venv` from `deps` and not from `trainer`. Both stages have a venv at that path, but the trainer's has pandas in it.

2. Docker Compose

Over 30 calls each, a cache miss takes 0.892 ms and a hit takes 0.135 ms, so a hit is 6.61 times faster. A miss costs two Redis round trips plus the model, while a hit costs one GET. The label is correct either way.

A Dockerfile builds one image and a container runs one process tree. Neither can say that this API needs a separate Redis next to it, on a shared network, that it can reach by name. Compose runs containers on one host. Kubernetes runs them across a cluster of nodes and also handles scheduling, self-healing and rolling updates.

3. Kubernetes Indexed Job

This is a separate batch job. A signup ledger is split into 8 partitions, each with a seeded number of bad rows. A row is bad if a required column is empty or the email address is malformed.

The cluster has 2 nodes with 2 CPUs each, so 4 in total. Each pod makes one pass over a few hundred rows with a regex, which does not fill a core, so it asks for 500m with the limit set to the same value. At 500m, parallelism 4 takes 2 CPUs, which is half the budget. The other half is needed for the system pods, since the control-plane node already asks for 850m and the worker for 100m. Parallelism 8 would ask for all 4 CPUs and leave nothing for them, so some pods would sit Pending. The job would take about the same time and I would be claiming a parallelism that never happened.

The manifest sets `completions: 8`, `parallelism: 4`, `completionMode: Indexed`, `restartPolicy: Never`, CPU requests equal to limits, and `POD_NAME` and `NODE_NAME` from the Downward API. Indexed mode gives each pod a number from 0 to 7, and the pod opens the partition file with that number.

All 8 partitions matched the counts my generator seeded in, on both rows and bad rows: 6464 rows and 333 bad.

With 3 nodes and 6 CPUs I would raise parallelism to 8. System overhead would be about 1.5 CPUs, leaving roughly 4.5 free, and 8 pods at 500m only need 4. That fits, and it turns the two waves into one.

4. Kubernetes Deployment

Self-healing. The ReplicaSet controller does this, not the Deployment controller. A Deployment manages ReplicaSets, one per revision, and each ReplicaSet manages its own pods. It counts the pods with the matching label, compares that to the number I asked for, saw 1 instead of 2, and made a replacement. The new pod has a different name because it is a new object, not a restart.

Rolling update. A poller sent 250 requests during the update. 249 of them worked, and both version strings showed up, so the switch happened while requests were still coming in. The one failure was right at the switch. `maxUnavailable: 0` makes sure a new pod is ready before an old one is stopped, but it does

not drain the old one, so a request can briefly reach a pod that has already closed its listener. A `preStop` hook fixes that. The result is 99.6% availability.

As a Job this would not have worked. You cannot change a Job's pod template after you create it, so `kubect1 set image` is rejected, and a new version would mean deleting the Job and making a new one with nothing serving in between.

Deployment vs. Job. The Question 3 Job checks 8 fixed partitions. The work is finite and each part runs once, so the pods should finish and stop. Self-healing would be wrong there, because bringing back a pod that already exited fine would redo work. The Deployment serves requests that never finish, so a pod exiting is always a problem and the replica count should go back up. Batch work needs to end and be counted. Serving work needs to stay up.